

# TokTickIT อธิบายแบบเข้าใจง่าย (Feynman Style)

หลักการไฟน์แมนคือ: ถ้าอธิบายเรื่องหนึ่งให้เด็กมัธยมเข้าใจไม่ได้ แปลว่าเรายังไม่เข้าใจมันจริง เอกสารนี้จะอธิบายทุกส่วนของ TokTickIT (ยกเว้น CSS/หน้าตา UI) โดยอิงจากโค้ดจริงในโปรเจกต์ ทุก concept จะมี **analogy (เปรียบเทียบ)** ก่อน แล้วค่อยโยงกลับไปโค้ด

## บทที่ 0 — ภาพรวมทั้งระบบ

ลองนึกภาพร้านอาหาร:

| ส่วนของร้าน                   | ส่วนของระบบ             | ในโปรเจกต์นี้คือ     |
|-------------------------------|-------------------------|----------------------|
| ลูกค้าสั่งอาหารหน้าเคาน์เตอร์ | Frontend (React)        | client/              |
| พนักงานรับออเดอร์ส่งเข้าครัว  | API (Express)           | server/src/app.ts    |
| ครัวที่ปรุงอาหารจริง          | Database logic (Prisma) | server/src/prisma.ts |
| ตู้เย็น/สต็อกวัตถุดิบ         | ฐานข้อมูล (PostgreSQL)  | Docker container     |

เวลาลูกค้า (ผู้ใช้) กด "Check System": React จะเดินไปเคาะประตูครัว (เรียก API) → Express รับคำขอ แล้วไปถามครัว (Prisma) → Prisma ไปหยิบของจากตู้เย็น (PostgreSQL) → ส่งต่อกลับมาเป็นทอดๆ จนถึงหน้าจอ

นี่คือ **"full-stack vertical slice"** ที่ lab สั่งให้ทำ: พิสูจน์ว่าทุกชั้นคุยกันได้จริง ไม่ใช่แค่ mock

## บทที่ 1 — Backend: Express REST API

### 1.1 Express คืออะไร (แบบเด็กมัธยมเข้าใจ)

Express เป็นเหมือน พนักงานต้อนรับที่ยืนอยู่หน้าประตู คอยฟังว่ามีใครมาเคาะประตูของไหน (URL ไหน) ด้วย method อะไร (GET, POST, ...) แล้ววิ่งไปทำงานที่กำหนดไว้ให้

โค้ดจริงจาก `server/src/app.ts` :

```
export function createApp(prisma = createPrismaClient()) {
  const app = express();

  app.use(cors({ origin: allowedOrigins() }));
  app.use(express.json());

  app.get("/api/health", (_req, res) => {
    res.json({ status: "ok", service: "TokTickIT API" });
  });

  app.get("/api/categories", async (_req, res) => {
    const categories = await prisma.category.findMany({
      orderBy: { id: "asc" },
    });
    res.json(categories);
  });
}
```

```
return app;
}
```

อ่านที่ละบรรทัดแบบ Feynman:

- `app.get("/api/health", (_req, res) => {...})` แปลว่า "ถ้ามีคนมาเคาะที่ประตู `/api/health` ด้วยวิธี **GET (แค่ขอ ไม่ได้ส่งของมาด้วย)** ให้ทำฟังก์ชันนี้" ฟังก์ชันนี้ไม่แตะฐานข้อมูลเลย มันแค่ตอบ "ผมยังมีชีวิตอยู่" — เหมือนเคาะประตูบ้านแล้วมีคนมารับ ไม่ได้แปลว่าครัวข้างในพร้อมทำอาหาร แค่แปลว่า "มีคนอยู่บ้าน"
- `app.get("/api/categories", async (_req, res) => {...})` อันนี้ต่างออกไป เพราะมันมี `async` และข้างในมันเรียก `prisma.category.findMany()` ซึ่งต้องไปเดินไปหยิบของจากตู้เย็นจริงๆ (query ฐานข้อมูลจริง) จึงต้อง `await` (รอ) ก่อนตอบกลับ
- `res.json(...)` แปลว่า "ตอบกลับเป็นข้อมูลรูปแบบ **JSON**" — เหมือนเขียนใบเสร็จเป็นภาษาที่ทั้งสองฝั่งเข้าใจตรงกัน

## 1.2 ทำไมต้องเป็น "Factory Pattern" ( `createApp()` แทนที่จะสร้าง `app` ตรงๆ )

นี่คือจุดที่หลายคนงงที่สุด ลองเทียบ:

- **แบบสร้างตรงๆ (ไม่ใช่ factory):** เหมือนเปิดร้านจริง มีไฟ มีลูกค้าเข้าออกจริง ถ้าอยากทดสอบว่า "ถ้าลูกค้าสั่งเมนู A จะได้ของถูกไหม" ต้องเปิดร้านจริงทุกครั้ง เสียเวลา เสียของจริง
- **แบบ Factory ( `createApp()` ):** เหมือนมี พิมพ์เขียวร้าน ที่สร้างร้านจำลองขึ้นมาได้ทุกครั้งที่ยากทดสอบ โดยไม่ต้องเปิดไฟจริง ไม่ต้องมีลูกค้าจริงเดินเข้ามา

โค้ด:

```
export function createApp(prisma = createPrismaClient()) {
  const app = express();
  // ...
  return app;
}
```

`createApp()` คืนค่า `app object` แปลว่า ที่ยังไม่ได้ `.listen()` (ยังไม่ได้เปิดไฟรอรับลูกค้าจริงบน network) เพราะฉะนั้น Supertest (เครื่องมือทดสอบ) เอา `app object` นี้ไปยิง request ปลอมใส่ได้เลยโดยไม่ต้องเปิด port จริง

เทียบกับ `server/src/server.ts` :

```
const port = Number(process.env.PORT ?? 3001);
createApp().listen(port, () => {
  console.log(`TokTickIT API listening on http://localhost:${port}`);
});
```

ไฟล์นี้ต่างหากที่ "เปิดไฟจริง" ( `.listen(port)` ) — คือตัวที่รันตอน `npm run dev` จริงๆ `app.ts` = พิมพ์เขียว, `server.ts` = ร้านที่เปิดไฟจริง

## 1.3 CORS คืออะไร ทำไมต้องมี

Browser มีกฎความปลอดภัยว่า หน้าเว็บที่โหลดจาก origin A จะยิง request ไปหา origin B ไม่ได้ เว้นแต่ B จะบอกอนุญาตชัดเจน (เหมือนบัตรเชิญทำงาน)

```
function allowedOrigins() {
  return (process.env.CLIENT_ORIGIN ?? "http://localhost:5173")
    .split(",")
    .map((origin) => origin.trim())
    .filter(Boolean);
}
```

CLIENT\_ORIGIN ในไฟล์ .env คือ "รายชื่อแบกที่มีบัตรเชิญ" เช่น http://localhost:5173 ถ้า React รันอยู่ที่ port อื่น (เช่น 5174 เพราะ 5173 ถูกใช้ไปแล้ว) แล้วไม่ได้อยู่ใน list นี้ Browser จะบล็อก request ทันที — นี่คือสาเหตุของบั๊ก "still a blank site" ที่เจอดอนพอร์ดชงกัน

โค้ดรองรับหลาย origin คั่นด้วย comma เพราะบางทีเรารันหลาย dev server พร้อมกัน (เช่นตอนเทียบกับโค้ดเพื่อน)

## บทที่ 2 — Database Layer: Prisma + PostgreSQL

### 2.1 PostgreSQL คืออะไร (ระดับเด็กมัธยม)

PostgreSQL คือ ตัวเก็บของที่มีลิ้นชักเป็นตาราง (table) แต่ละลิ้นชักมีช่องเก็บของหลายแถว (row) แต่ละแถวมีข้อมูลตามคอลัมน์ที่กำหนดไว้ตายตัว

### 2.2 Prisma คืออะไร — ทำไมไม่เขียน SQL ตรงๆ

ถ้าไม่มี Prisma เราต้องเขียนคำสั่งแบบนี้ตรงๆ:

```
SELECT * FROM "Category" ORDER BY id ASC;
```

Prisma คือ **ล่ามแปลภาษา** ที่แปลงโค้ด JavaScript/TypeScript ให้กลายเป็น SQL ให้อัตโนมัติ และที่สำคัญกว่านั้น มันทำให้ TypeScript **รู้ล่วงหน้า** ว่าตาราง Category มีคอลัมน์อะไรบ้าง (auto-complete และ type-check ตอนเขียนโค้ด — พิมพ์ผิดคอลัมน์มี error ทันทีตอน compile ไม่ต้องรอรันจริง)

### 2.3 Schema — พิมพ์เขียวของตาราง

server/prisma/schema.prisma :

```
model Category {
  id      Int      @id @default(autoincrement())
  name    String   @unique
  description String
  createdAt DateTime @default(now())
}
```

อ่านทีละบรรทัด:

- id Int @id @default(autoincrement()) — "เลขลำดับ ให้ Postgres นับเพิ่มเองอัตโนมัติ ห้ามซ้ำ (คือ primary key)"
- name String @unique — "ชื่อ ต้องไม่ซ้ำกันในตารางนี้" ( @unique คือกฎที่บังคับระดับฐานข้อมูลจริง ไม่ใช่แค่เช็คฝั่ง JavaScript — ถ้าพยายามยัดชื่อซ้ำ Postgres จะปฏิเสธเอง)
- createdAt DateTime @default(now()) — "เวลานับที่กลง ให้ Postgres จดเวลาปัจจุบันให้เองถ้าไม่ได้ระบุมา"

**Schema** นี้ไม่ได้สร้างตารางจริงในฐานข้อมูลทันทีที่เขียน — มันเป็นแค่พิมพ์เขียว ต้องมี "migration" มาสั่งให้ Postgres สร้างจริง

## 2.4 Migration คืออะไร

Migration คือ ใบสั่งงานช่างต่อเติมบ้าน ที่บอกทีละขั้นตอนว่าต้องทุบผนังไหน เติมหองไหน เพื่อให้บ้าน (ฐานข้อมูล) เปลี่ยนจากสภาพเดิมเป็นสภาพใหม่ตาม schema

รันคำสั่ง:

```
npx prisma migrate dev --name add_category_table
```

Prisma จะ diff schema เก่ากับใหม่ แล้วเขียนไฟล์ SQL ให้เองใน `prisma/migrations/.../migration.sql` :

```
-- AlterTable
ALTER TABLE "Category" ADD COLUMN "createdAt" TIMESTAMPTZ(3) NOT NULL DEFAULT
CURRENT_TIMESTAMP;
```

จุดสำคัญ: **migration ถูกเก็บเป็นไฟล์ ประวัติศาสตร์การเปลี่ยนแปลงตารางทั้งหมด** ใครก็ตามที่ clone repo นี้ แล้วรัน `npx prisma migrate deploy` จะได้ตารางหน้าตาเหมือนกันเป๊ะ ไม่ว่าจะเป็นเครื่องไหน

## 2.5 Driver Adapter — ทำไม Prisma 7 ต้องมีขั้นตอนพิเศษ

Prisma รุ่นเก่าอ่าน `DATABASE_URL` เองอัตโนมัติ แต่ Prisma 7 เปลี่ยนกฎ บังคับให้ระบุ **adapter** เอง เหมือนเปลี่ยนจาก "โทรศัพท์มือถือในตัว โทรได้เลย" เป็น "ต้องเสียบซิมการ์ด (adapter) เข้าเครื่องก่อนถึงจะโทรได้"

`server/src/prisma.ts` :

```
const connectionString = process.env.DATABASE_URL;

if (!connectionString) {
  throw new Error("DATABASE_URL is not set. Copy server/.env.example to server/.env.");
}

const adapter = new PrismaPg({ connectionString });
export const prisma = new PrismaClient({ adapter });

export function createPrismaClient() {
  const newAdapter = new PrismaPg({ connectionString: connectionString! });
  return new PrismaClient({ adapter: newAdapter });
}
```

- `export const prisma` — ตัวแชร์ตัวเดียวใช้ทั้งแอปตอนรันจริง (เพราะแต่ละ `PrismaClient` เปิด connection pool ไปที่ Postgres ถ้าสร้างใหม่ทุก request จะเปิด connection ท่วมจน Postgres ปฏิเสธ — เหมือนเปิดสายโทรศัพท์ใหม่ทุกประโยคที่พูด)
- `createPrismaClient()` — factory แยกไว้ให้ test สร้าง instance ใหม่ของตัวเองได้ ไม่ต้องแชร์ state กับ production

## 2.6 Seed คืออะไร

Seed คือ การหว่านเมล็ดข้อมูลตั้งต้น ลงตารางเปล่าๆ ให้แอปมีของให้แสดงตั้งแต่แรก

`server/prisma/seed.ts` :

```

async function main() {
  const categories = [
    { name: 'Account and Access', description: '...' },
    { name: 'Hardware', description: '...' },
    { name: 'Software', description: '...' },
    { name: 'Network', description: '...' },
  ]

  for (const cat of categories) {
    await prisma.category.upsert({
      where: { name: cat.name },
      update: cat,
      create: cat,
    })
  }

  console.log('Seeded 4 categories')
}

```

จุดสำคัญที่สุดคือ `upsert` — ย่อมาจาก **UP**date + **inS**ERT:

- ถ้าเจอแถวที่ `name` ตรงกับ `where` อยู่แล้ว → แก้ไข (update) แถวนั้นให้ตรงตามข้อมูลใหม่
- ถ้ายังไม่มี → สร้างใหม่ (create)

เปรียบเทียบง่ายๆ: เหมือนเช็คอินโรงแรม ถ้าชื่อคุณอยู่ในระบบแล้วก็แค่อัปเดตข้อมูล ไม่ใช่จองห้องใหม่ซ้ำ **นี่คือเหตุผลที่รัน seed กี่รอบก็ไม่มีข้อมูลซ้ำ (idempotent)** — ต่างจากการใช้ `create` ตรงๆ ที่จะ error เพราะ `name` เป็น `@unique` (รันซ้ำจะชนกฎ unique) หรือถ้าใช้ `deleteMany()` ก่อนสร้างใหม่ก็ทำได้เหมือนกัน แต่ `upsert` ปลอดภัยกว่าเพราะไม่ต้องลบของเดิมทิ้งก่อน (ถ้ามีตารางอื่นอ้างอิง id ของ category อยู่ การลบแล้วสร้างใหม่ จะทำให้ id เปลี่ยน ของที่อ้างอิงอยู่จะหลุดผูกไปด้วย)

## บทที่ 3 — Testing: Supertest + Vitest

### 3.1 ทำไมต้องมี Test

Test คือ หน้าที่ QA ที่คอยตรวจสอบให้เราแทน ทุกครั้งที่โค้ดเปลี่ยนแปลง โดยไม่ต้องนั่งทดสอบมือเองซ้ำๆ

### 3.2 Supertest — ทดสอบฝั่ง Backend

Supertest จำลองการยิง HTTP request ใส่ app โดยไม่ต้องเปิด port จริง (ใช้ `createApp()` ที่อธิบายไปแล้ว)

server/tests/lab-01/API-01.health.test.ts :

```

describe("API-01 GET /api/health", () => {
  it("returns HTTP 200", async () => {
    const response = await request(createApp()).get("/api/health");
    expect(response.status).toBe(200);
  });

  it("reports status ok and the service name", async () => {
    const response = await request(createApp()).get("/api/health");
    expect(response.body).toEqual({
      status: "ok",
      service: "TokTickIT API",
    });
  });
});

```

```
});
});
});
```

อ่านแบบ Feynman:

- `request(createApp())` — "สร้างร้านจำลอง แล้วส่งลูกค้าปลอมเดินเข้าไป"
- `.get("/api/health")` — "ลูกค้าปลอมคนนี้เดินไปเคาะประตู /api/health"
- `expect(response.status).toBe(200)` — "ต้องได้ใบเสร็จรหัส 200 (สำเร็จ) เท่านั้น ถ้าไม่ใช่ = สอดตก"
- `expect(response.body).toEqual({...})` — "เนื้อหาใบเสร็จต้องตรงเป๊ะกับที่คาดไว้"

`API-02.categories.test.ts` ทดสอบซับซ้อนกว่าเพราะแต่ละฐานข้อมูลจริง (ไม่ mock) — มันพิสูจน์ว่า seed ทำงานจริง ไม่ใช่แค่ได้ถูกต้องบนกระดาษ

### 3.3 Vitest + Testing Library — ทดสอบฝั่ง Frontend

ฝั่ง React ทดสอบยากกว่าเพราะต้อง "แกล้งเป็น browser" (jsdom) และต้อง "แกล้งเป็นเครือข่าย" เพราะไม่ยอมให้ test จริงไปถึง API จริงทุกครั้ง (ช้า, ไม่เสถียร, ต้องมี server เปิดอยู่)

`client/tests/lab-01/UI-02.categories.test.tsx` :

```
it('loads and displays all four categories when user clicks Check System', async () => {
  vi.spyOn(globalThis, 'fetch').mockResolvedValueOnce(
    Response.json({ status: 'ok', service: 'TokTickIT API' }),
  )
  vi.spyOn(globalThis, 'fetch').mockResolvedValueOnce(
    Response.json(mockCategories),
  )
  const user = userEvent.setup()

  render(<App />)
  await user.click(screen.getByRole('button', { name: /Check System/i }))

  await waitFor(() => {
    expect(screen.getByText('Hardware')).toBeInTheDocument()
  })
})
```

อ่านแบบ Feynman:

- `vi.spyOn(globalThis, 'fetch').mockResolvedValueOnce(...)` — "แกล้งปลอมคำตอบของ `fetch()` " แปลว่าเวลาใดก็ตามที่เรียก `fetch(...)` มันจะไม่ได้ยิง network จริง แต่ได้คำตอบปลอมที่เราเตรียมไว้ทันที ที่ทำไมต้องเรียก `mockResolvedValueOnce` สองครั้ง? เพราะ `App.tsx` เรียก `fetch` สองรอบติดกัน: รอบแรกคือ health check รอบสองคือ categories — mock ตัวแรกตอบคำถามที่หนึ่ง ตัวที่สองตอบคำถามที่สอง (เรียงตามลำดับที่ใดก็ตามที่เรียก)
- `render(<App />)` — "วาด component ทั้งหมดลงใน DOM ปลอม (jsdom) เหมือนเปิดเบราว์เซอร์จำลอง"
- `userEvent.setup() + user.click(...)` — "จำลองนิ้วมนุษย์กดปุ่มจริงๆ" (ต่างจาก `fireEvent.click` ตรงที่ `userEvent` จำลองพฤติกรรมจริงของ browser ครบกว่า เช่น focus, hover ก่อนคลิก)
- `waitFor(() => {...})` — "รอกันกว่า promise ข้างในจะ resolve" จำเป็นเพราะ React ต้อง re-render หลัง state เปลี่ยนแบบ async (`fetch` เป็น asynchronous ผลลัพธ์ไม่มาในทันที)

### 3.4 Mock ทำไมสำคัญ — Test-Doubling

ลองเทียบ: ถ้าอยากซ่อมขั้วรถแต่ไม่อยากเสี่ยงชนคนจริงบนถนนจริง เราซ่อมในสนามจำลองก่อน `mock` คือสนามจำลองของโค้ด — มันทำให้ `test` เร็ว (ไม่ต้องรอ `network` จริง), เสถียร (ไม่พังเพราะเน็ตล่มหรือ `server` ไม่ได้เปิด), และ แยกส่วนทดสอบ (ทดสอบแค่ตรรกะของ `React` ไม่ปนกับว่า `backend` ทำงานถูกไหม — `backend` มี `test` ของตัวเองแยกต่างหากคือ `Supertest test`)

## บทที่ 4 — Frontend Logic (ไม่รวม CSS)

### 4.1 React Component คืออะไร

Component คือ แม่พิมพ์ที่รับข้อมูลเข้า (`state`) แล้วคายหน้าตาที่ควรแสดงออกมา ทุกครั้งที่ `state` เปลี่ยน `React` จะ "พิมพ์ใหม่" (`re-render`) อัตโนมัติ

### 4.2 State — ความจำของ Component

`client/src/App.tsx` :

```
type CheckState =
  | { phase: 'idle' }
  | { phase: 'loading' }
  | { phase: 'online'; service: string; categories: Category[] }
  | { phase: 'offline'; message: string }

function App() {
  const [check, setCheck] = useState<CheckState>({ phase: 'idle' })
  // ...
}
```

นี่คือเทคนิคที่เรียกว่า **discriminated union** (union ที่แยกแยะได้ด้วย field เดียว คือ `phase`) เปรียบเหมือนไฟจราจร 4 สี: `idle` (ยังไม่กด), `loading` (กำลังตรวจสอบ), `online` (เช็คแล้วระบบพร้อม พร้อมข้อมูล `categories` แนบมาด้วย), `offline` (เช็คแล้วพัง พร้อมข้อความ `error` แนบมาด้วย)

จุดเด็ดของแบบนี้คือ TypeScript บังคับ ว่าถ้า `check.phase === 'online'` โค้ดถึงจะรู้ว่า `check.categories` มีอยู่จริง (type narrowing) — ป้องกัน bug แบบ "ลืมเช็คว่ามีข้อมูลก่อนใช้"

### 4.3 useState คืออะไร (แบบง่ายที่สุด)

`useState<CheckState>({ phase: 'idle' })` คืนค่าออกมาสองตัว:

1. `check` — ค่าปัจจุบัน (เหมือนอ่านค่าจากกระดานชนวน)
2. `setCheck` — ฟังก์ชันเปลี่ยนค่า ที่จะสั่งให้ `React` วาดหน้าจอใหม่ทันทีที่เปลี่ยน

ข้อควรรู้: ห้ามแก้ `state` ตรงๆ (เช่น `check.phase = 'loading'`) เพราะ `React` จะไม่รู้ว่ามันเปลี่ยน ต้องเรียก `setCheck({...})` เท่านั้นแอปถึงจะ `re-render`

### 4.4 การเรียก API จาก Frontend

`client/src/api.ts` :

```
export async function fetchHealth(): Promise<HealthResponse> {
  const response = await fetch(`${API_URL}/api/health`)
  if (!response.ok) {
    throw new Error(`TokTickIT API responded with ${response.status}`)
  }
}
```

```

    }
    return (await response.json()) as HealthResponse
  }
}

```

`fetch(...)` คือ "ยื่นของจดหมายไปที่ประตุนั้น แล้วรอคำตอบ" — เป็น async เพราะการเดินทางไป-กลับ ผ่านเครือข่ายใช้เวลา ไม่ใช่ทันที (เหมือนส่งจดหมายจริง ไม่ใช่ตะโกนคุยกันในห้องเดียว)

`response.ok` เช็คค่า status code อยู่ในช่วง 200-299 โหม (สำเร็จ) ถ้าไม่ใช่ (เช่น 500) จะ `throw error` ออกไปทันที — จุดนี้สำคัญเพราะ `fetch()` เอง **ไม่ throw error ให้อัตโนมัติ** ถ้า server ตอบ 500 (มัน throw เฉพาะตอนเน็ตล่มจริงๆ เช่น DNS หาไม่เจอ) โค้ดส่วนนี้จึงต้องเช็คเองเพื่อให้ทั้งสองเคสพังแบบเดียวกัน

## 4.5 การเชื่อมทุกอย่างเข้าด้วยกัน

client/src/App.tsx :

```

async function handleCheckSystem() {
  setCheck({ phase: 'loading' })
  try {
    await fetchHealth()
    const categories = await fetchCategories()
    setCheck({ phase: 'online', service: 'TokTickIT API', categories })
  } catch {
    setCheck({
      phase: 'offline',
      message: 'Unable to connect to TokTickIT API',
    })
  }
}

```

อ่านเป็นเรื่องราว:

- กดปุ่ม → บอก state ว่า "กำลังโหลด" ทันที (UI เปลี่ยนเป็น spinner ทันที ผู้ใช้รู้ว่ามันทำงานอยู่)
- `await fetchHealth()` — ไปถามก่อนว่า "ระบบยังอยู่ไหม" ถ้าล้มจะกระโดดไป `catch` ทันที
- ถ้าผ่าน ไปถามต่อ `fetchCategories()` — เอารายการหมวดหมู่มา
- ทั้งสองผ่านหมด → `setCheck({phase: 'online', ...})` → UI เปลี่ยนไปแสดงการ์ด categories
- ถ้าตรงไหนพัง (network error หรือ non-2xx) → `catch` จับได้หมด (เพราะทั้ง `fetchHealth` และ `fetchCategories` throw error แบบเดียวกันตามที่อธิบายในข้อ 4.4) → `setCheck({phase: 'offline', ...})`

นี่คือเหตุผลที่ทั้งระบบ error handling ใช้แค่ `try/catch` เดียวจบ — เพราะออกแบบให้ **ทุก failure มาในรูปแบบเดียวกัน** (`throw Error`) ไม่ต้องแยก case พิเศษ

## 4.6 Conditional Rendering (ไม่เกี่ยวกับ CSS)

```

{check.phase === 'offline' && (
  <div className="alert alert-danger small mb-0" role="alert">
    {check.message}
  </div>
)}

```



`{condition && <JSX/>}` คือ "ถ้า **condition** เป็นจริง ให้แสดง **JSX** นี้ ถ้าไม่ใช่ไม่แสดงอะไรเลย" (มาจากพฤติกรรม JavaScript: `true && X` ได้ `X`, `false && X` ได้ `false` ซึ่ง React จะไม่ render `false`)

`role="alert"` ไม่ใช่ CSS แต่เป็น **accessibility attribute** — บอก screen reader (สำหรับผู้พิการทางสายตา) ว่า "นี่คือข้อความแจ้งเตือนสำคัญ อ่านออกเสียงทันทีที่ปรากฏ" และเป็นสิ่งที่ test ใช้นค้นหา element ด้วย (`screen.getByRole('alert')`) ในบทที่ 3)

---

## บทที่ 5 — Git Workflow ที่ใช้ในโปรเจกต์นี้

### 5.1 ทำไมต้องมี Branch

Branch คือ **สำเนาโลกคู่ขนาน** ที่แก้ไขโค้ดได้อิสระโดยไม่กระทบโค้ดหลัก (`main`) จนกว่าจะพร้อมเอากลับมารวม

รูปแบบที่ใช้ในโปรเจกต์นี้:

```
main
├─ lab1-staging
│   ├── feature/1-project-foundation
│   ├── feature/2-health-check
│   └─ feature/3-categories
```

แต่ละ `feature/*` คือ 1 Issue งาน — แยกกันทำเพื่อให้ตรวจสอบและ review ที่ละก้อนได้ง่าย ไม่ปนกันจนตามไม่ทันว่าอะไรทำอะไร

### 5.2 Commit คืออะไร

Commit คือ **สแนปช็อตของโค้ด ณ ขณะหนึ่ง** พร้อมข้อความอธิบายว่าทำไมถึงเปลี่ยน เหมือน checkpoint ในเกม — ย้อนกลับไปจุดไหนก็ได้ถ้าพลาด

กฎที่ดี: commit message บอก "ทำไม" มากกว่า "ทำอะไร" (เพราะ diff บอกอยู่แล้วว่าทำอะไร) เช่น `"Allow CLIENT_ORIGIN to list several origins"` ดีกว่า `"edit app.ts"`

### 5.3 Pull Request (PR) คืออะไร

PR คือ **การขออนุญาตเอา branch งานของเรากลับไปรวมกับ branch หลัก** พร้อมเปิดพื้นที่ให้คนอื่น (reviewer) เข้ามาอ่าน diff และคอมเมนต์ก่อนอนุมัติ — เหมือนส่งการบ้านให้เพื่อนตรวจก่อนส่งจริง

### 5.4 ทำไมบางที force-push ถึงจำเป็น (แต่ต้องระวัง)

`git push --force` คือ **เขียนประวัติศาสตร์ทับของเก่าบน remote** ใช้ตอนที่เรา `reset` หรือ `commit --amend` แก้ประวัติใน local แล้วต้องบังคับให้ remote hard ตามด้วย — อันตรายเพราะถ้ามีคนอื่น push งานลงมาแล้วในระหว่างนั้น งานของเราจะหายไปเลย เพราะฉะนั้นควรใช้กับ branch ของตัวเองที่ไม่มีคนแตะร่วม และควรถามยืนยันก่อนเสมอ

---

## สรุปภาพรวมทั้งหมด (Feynman Recap)

ลองอธิบายให้คนที่ไม่รู้อะไรเลยฟังแบบ 1 นาที:

"เรามีเว็บที่กดปุ่มแล้วเช็ควาระบบหลังบ้านทำงานไหม (Express ตอบว่า 'ok') แล้วไปขอรายการหมวดหมู่ จากฐานข้อมูลจริง (Prisma แปลคำสั่งเป็นภาษา SQL ให้ Postgres) ทุกอย่างถูกทดสอบด้วยหุ่นยนต์ปลอม (Supertest ปลอมเป็นลูกค้าเดินเข้าประตู backend, Vitest+fetch mock ปลอมเป็นเครือข่ายฝั่ง frontend) และงานทุกชิ้นถูกแยกเป็นกิ่งไม้ (branch) ของตัวเอง ก่อนจะขอรวมกลับเข้าลำต้นหลัก (main) ผ่าน PR ที่มีคนตรวจ"

ถ้าอธิบายประโยคนี้ให้คนอื่นฟังแล้วเขายกหน้าเข้าใจ แปลว่าเราเข้าใจงานนี้จริงแล้ว